R2P Program Unit 3 - Lesson 1

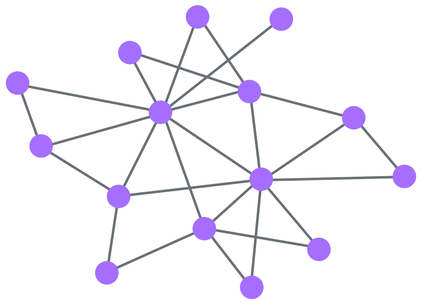
Transpilation made good

Meltem Tolunay
Quantum Algorithm Engineer
IBM Quantum

IBM **Quantum**

IBM

Qiskit Pattern:
The anatomy of a quantum algorithm



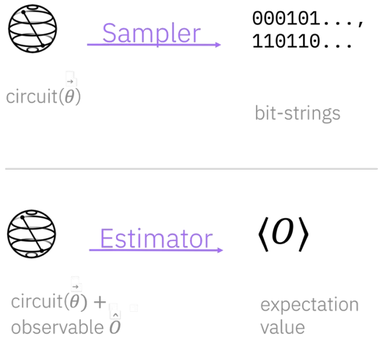
01

Map problem instance to quantum circuits and operators

```
PassManager([UnitarySynthesis(),
             BasisTranslator(),
             EnlargeWithAncilla(),
             AlSwap(),
             Collect1qRuns(),
             Optimize1qGates(),
             Collect2qBlocks(),
             ConsolidateBlocks()])
```

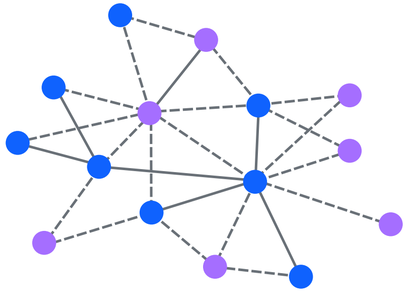
02

Optimize for target hardware execution



03

Execute via Qiskit Runtime



04

Result processing

Map

Optimize

Execute

Post-Process

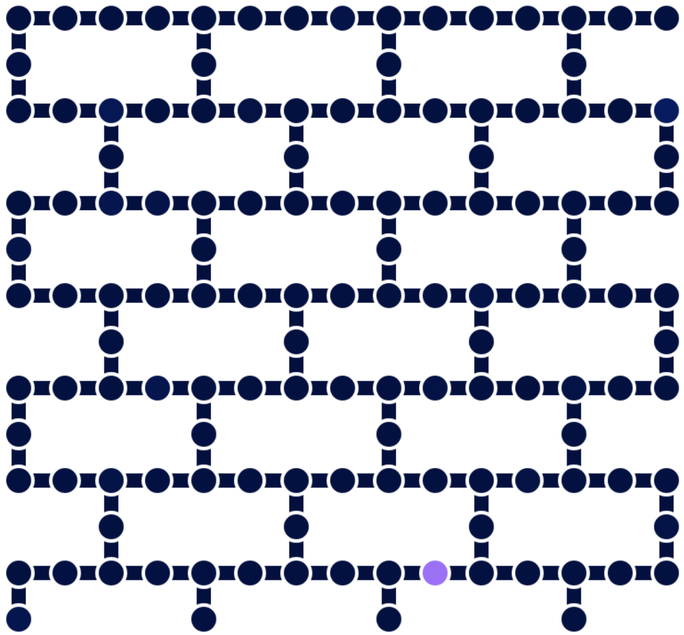
Optimizing quantum circuits for execution

Step 2

(in the Qiskit pattern framework)

Optimize problem for quantum execution.

```
PassManager([UnitarySynthesis(),
             BasisTranslator(),
             EnlargeWithAncilla(),
             AISwap(),
             Collect1qRuns(),
             Optimize1qGates(),
             Collect2qBlocks(),
             ConsolidateBlocks()])
```



Transpilation rewrites circuits from Step 1 to circuits that can be run on quantum hardware, matching topology and optimizing the circuit instructions for execution on noisy quantum computers.

Gates of the circuit must be transformed to the single-and two-qubit **basis gates** of the underlying quantum hardware.

Limited Connectivity means SWAP gates must be (efficiently) inserted into circuit to route quantum information around lattice

The output of transpilation is called an **Instruction Set Architecture (ISA) circuit**

Step 2:

Optimize quantum inputs

Qiskit transpiler

Convert abstract circuits to ISA circuits that respect the constraints of the target hardware. Optimizes circuit performance.

Pass Manager

Flexibly configure what optimizations are applied to your circuits.

In Qiskit 1.0:

Obtain shorter depth circuits with faster transpilation times.

Industry-leading transpiler performance

16x faster binding and transpiling

Compared to Qiskit 0.33

23% fewer 2Q gates

Compared to Tket 1.21
Avg. of six 100-qubit algorithms

Instruction set architecture

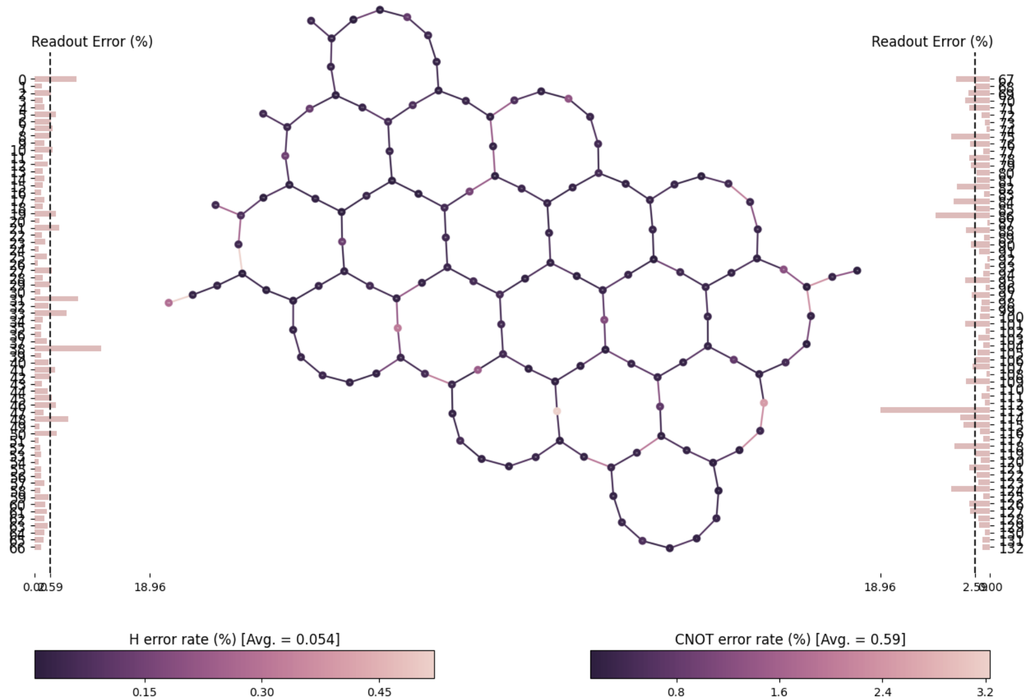
Quantum circuits come in many different shapes and forms; however, not all of them can run directly on quantum hardware.

To run, circuits must conform to the hardware's specific Instruction Set Architecture (ISA). This kind of circuits are known as ISA circuits.

ISA constraints come in the form of:

1. Gate types
2. Qubit connectivity

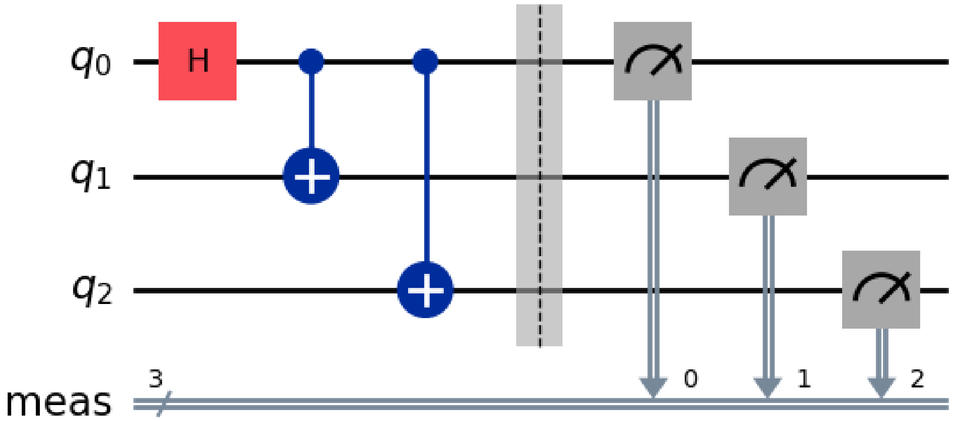
```python
from qiskit_ibm_runtime import QiskitRuntimeService
from qiskit.visualization import plot_error_map

service = QiskitRuntimeService()
backend = service.least_busy()

plot_error_map(backend, show_title=False)
```

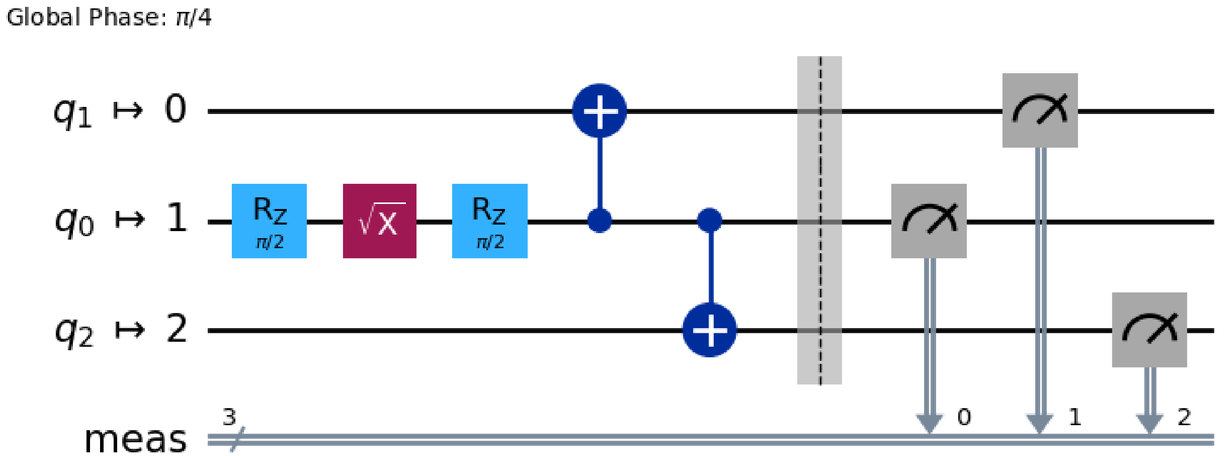


Transpilation

An ISA is said to be universal if it can express any operation possible on a quantum computer.

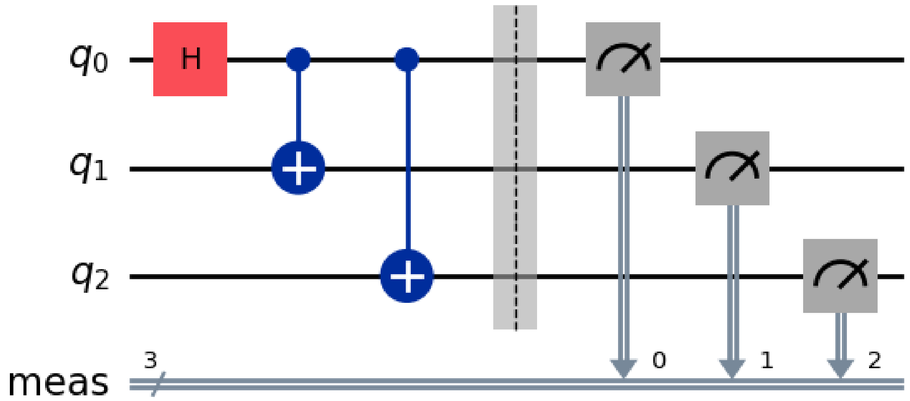
The process of expressing a quantum circuit in ISA form is known as transpilation, since the (circuit) level of abstraction is preserved.



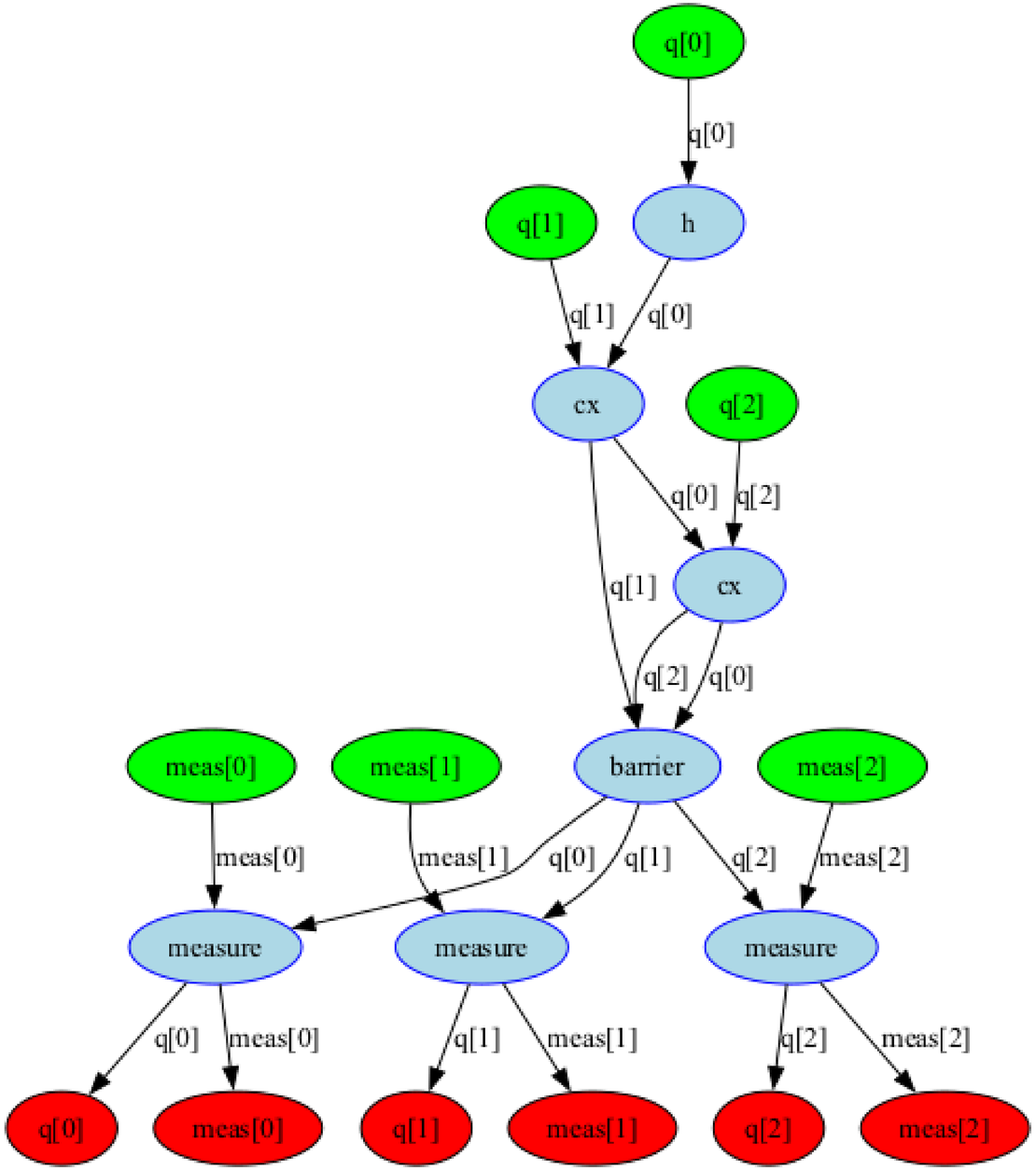


Circuit representation

To analyze and perform circuit transformations, Qiskit's transpiler uses a graph-based data structure internally known as DAGCircuit.




```python
from qiskit.converters import circuit_to_dag

dag = circuit_to_dag(circuit)
dag.draw()
```

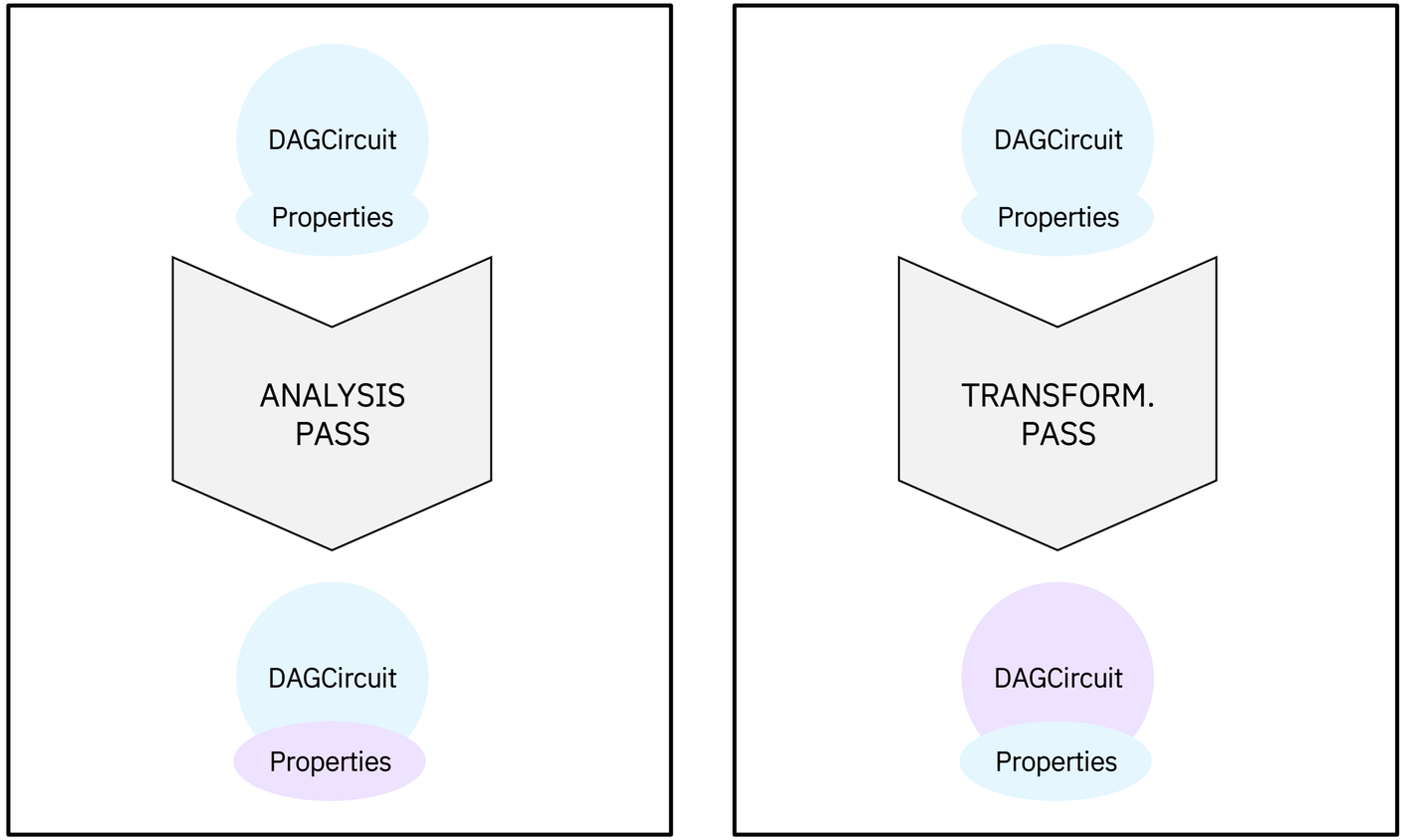


Transpiler architecture

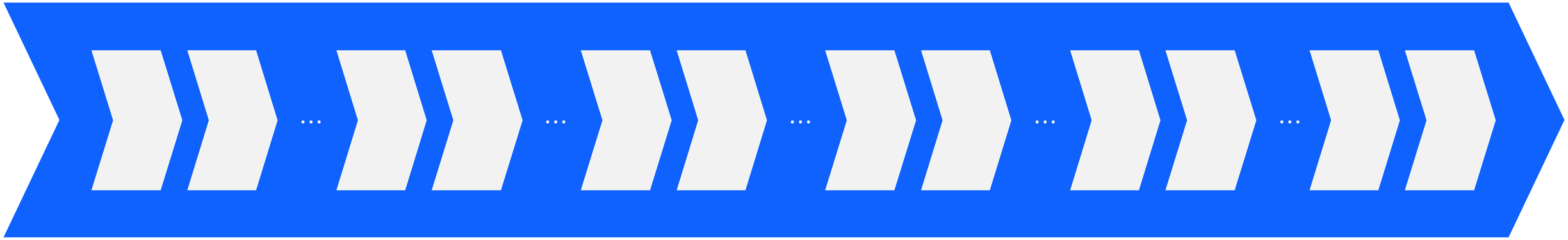
Actions on a circuit are performed by objects known as transpiler passes.

Transpiler passes come in two types:

1. Analysis passes
2. Transformations passes

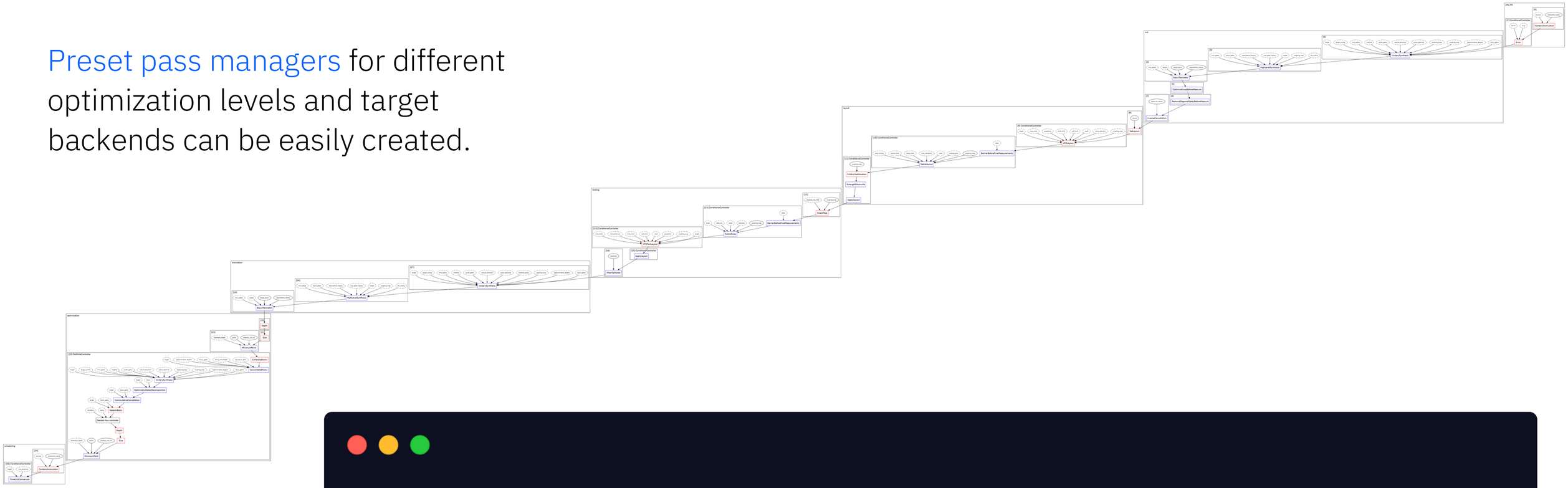


PASS MANAGER



Transpiler pipeline

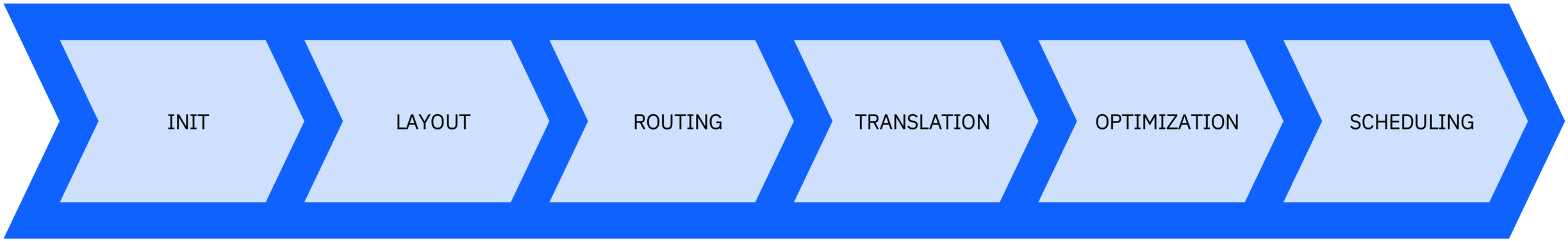
Preset pass managers for different optimization levels and target backends can be easily created.



```
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager

pm = generate_preset_pass_manager(optimization_level=3, backend=backend)
pm.draw()
```

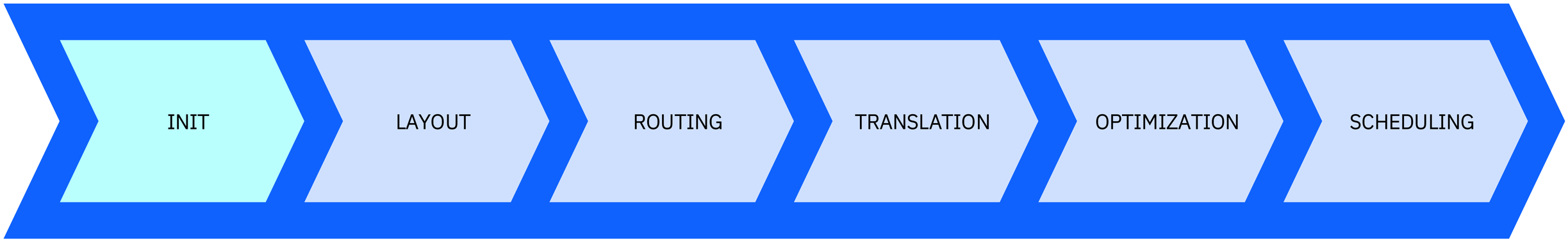
Transpiler stages

STAGED PASS MANAGER



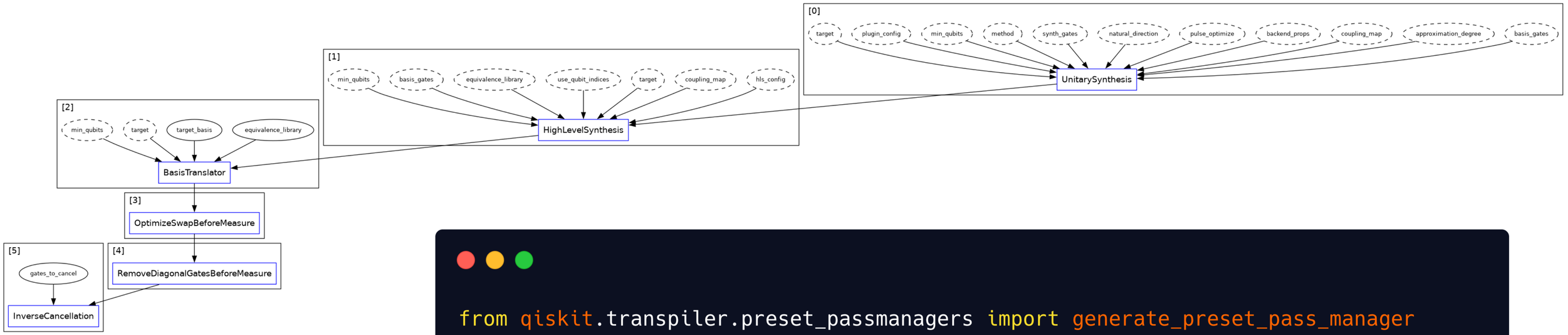
Transpiler stages

STAGED PASS MANAGER



Init stage

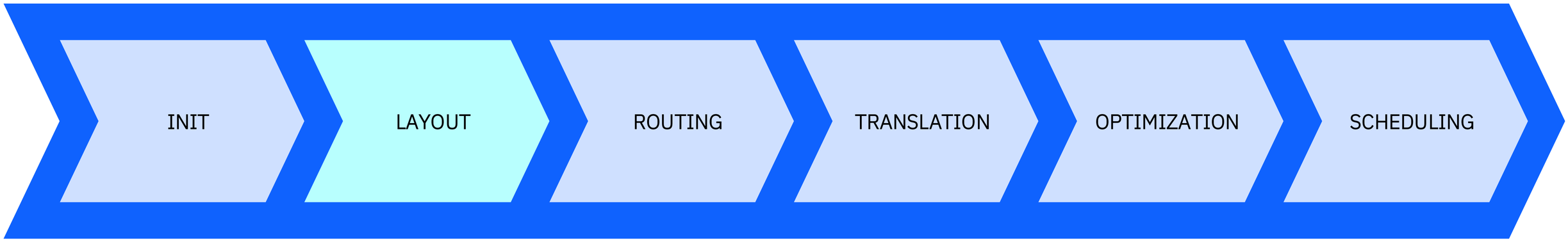
1. Unitary synthesis
2. High level synthesis
3. Basis translation
4. Optimize SWAP before measurement
5. Remove diagonal gates before measurements
6. Inverse cancelation




```python
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager

pm = generate_preset_pass_manager(optimization_level=3, backend=backend)
pm.init.draw()
```

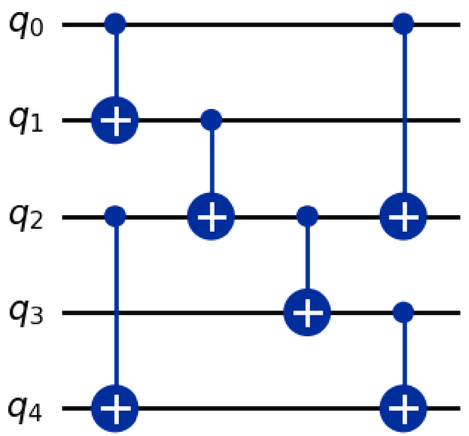
Transpiler stages

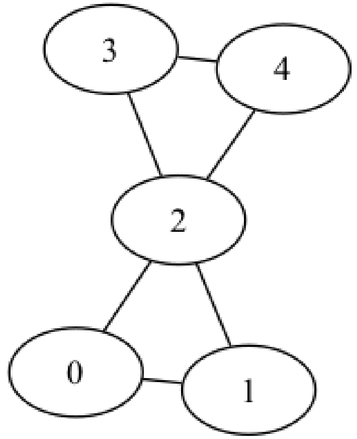
STAGED PASS MANAGER

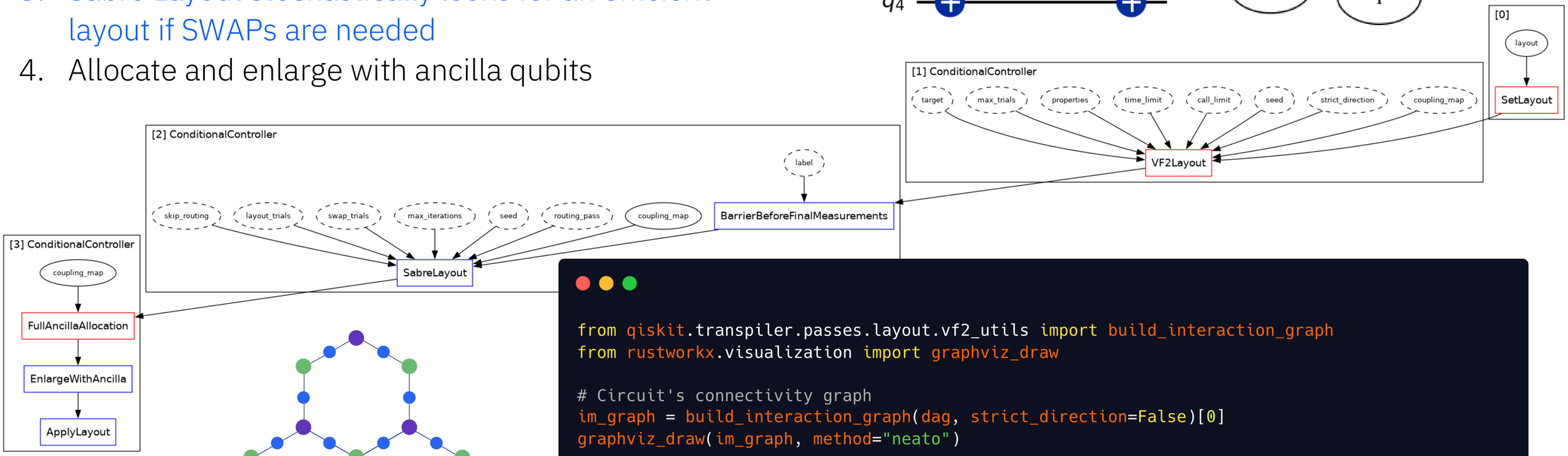


Layout stage

1. Set Layout if custom layout is specified manually
2. VF2 Layout looks for the best isomorphic subgraph in the backend for the circuit connectivity
3. Sabre Layout stochastically looks for an efficient layout if SWAPs are needed
4. Allocate and enlarge with ancilla qubits



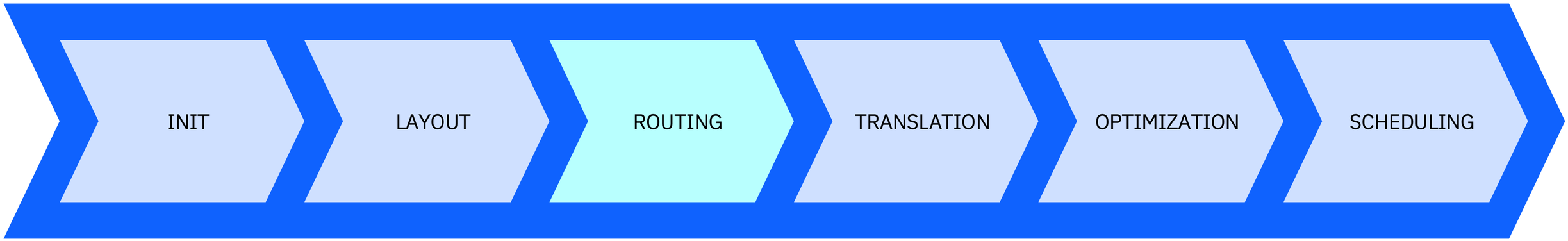





```python
from qiskit.transpiler.passes.layout.vf2_utils import build_interaction_graph
from rustworkx.visualization import graphviz_draw

# Circuit's connectivity graph
im_graph = build_interaction_graph(dag, strict_direction=False)[0]
graphviz_draw(im_graph, method="neato")

# Backend's connectivity graph
graphviz_draw(backend.coupling_map.graph.to_undirected(multigraph=False), method="neato")
```

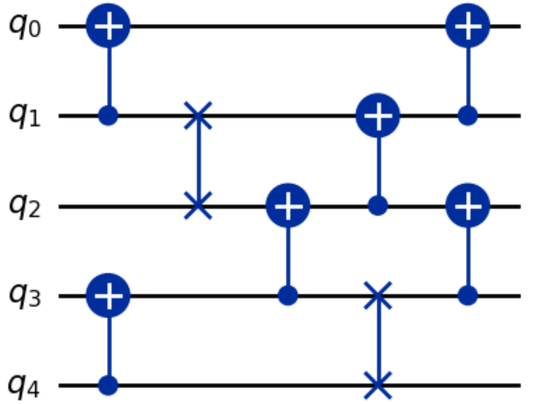
Transpiler stages

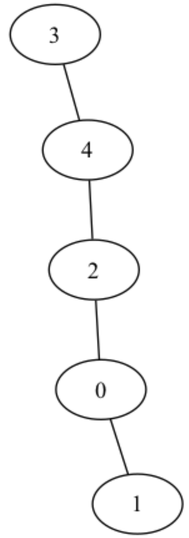
STAGED PASS MANAGER

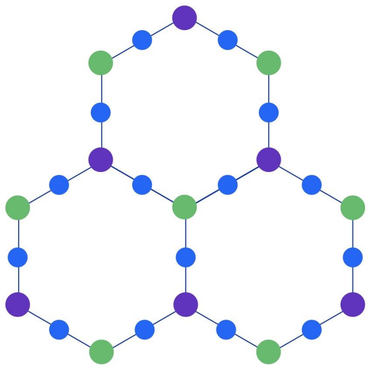


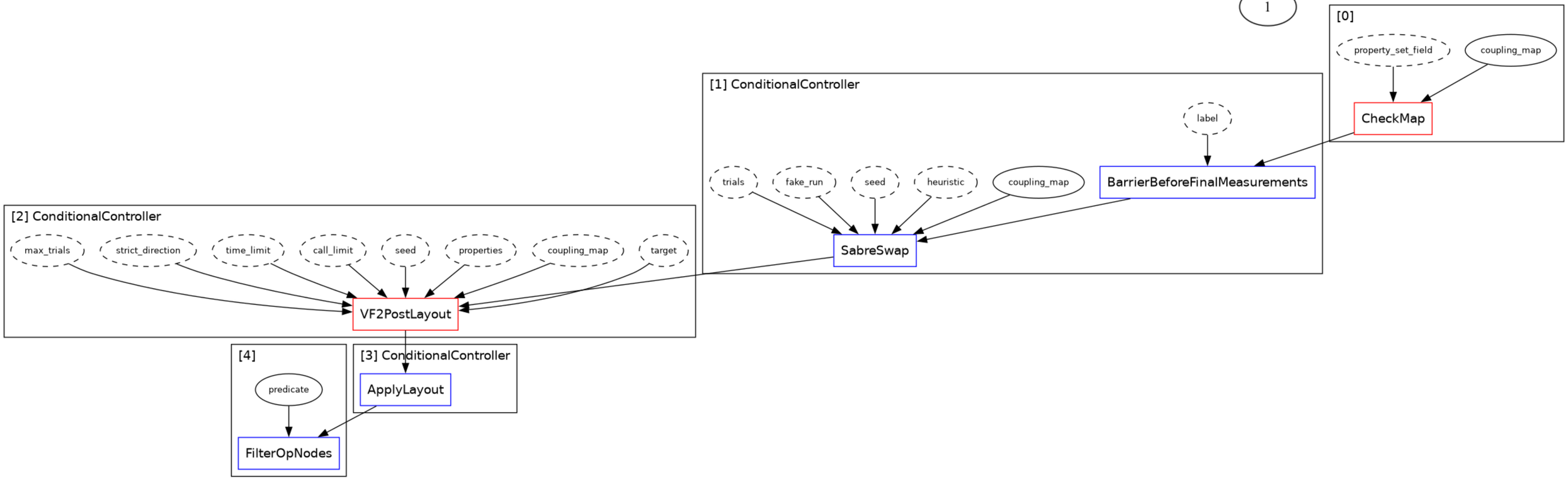
Routing stage

1. Check if routing is needed
2. Saber Swap
3. VF2 Post Layout to find all possible isomorphic subgraphs after SWAPs are inserted and choose the best one in terms of error rates.



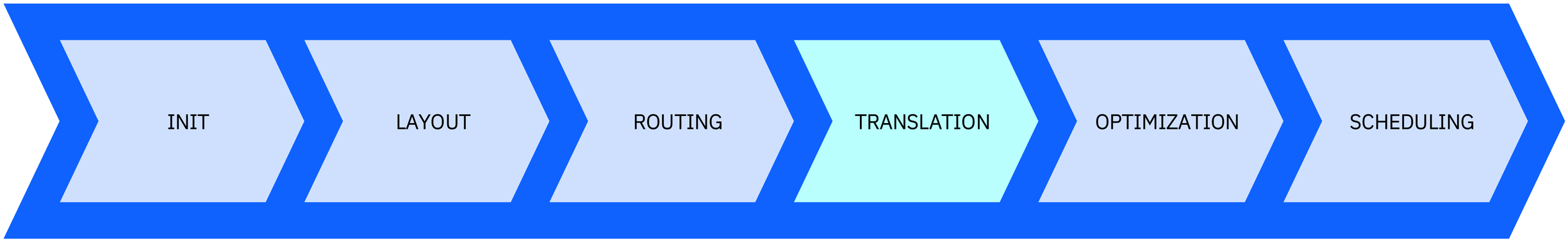






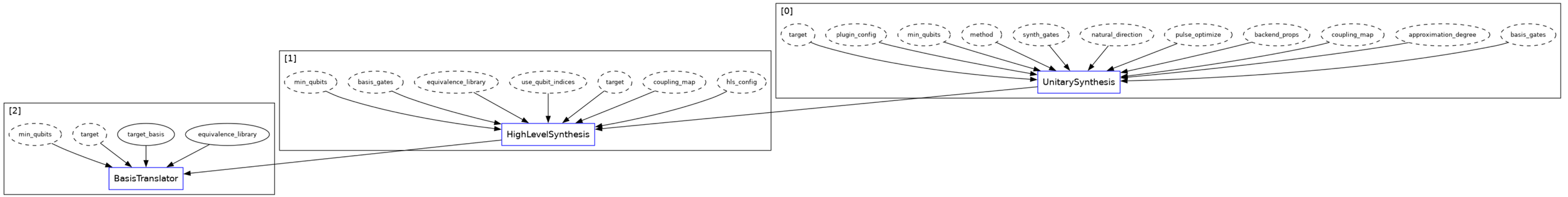
Transpiler stages

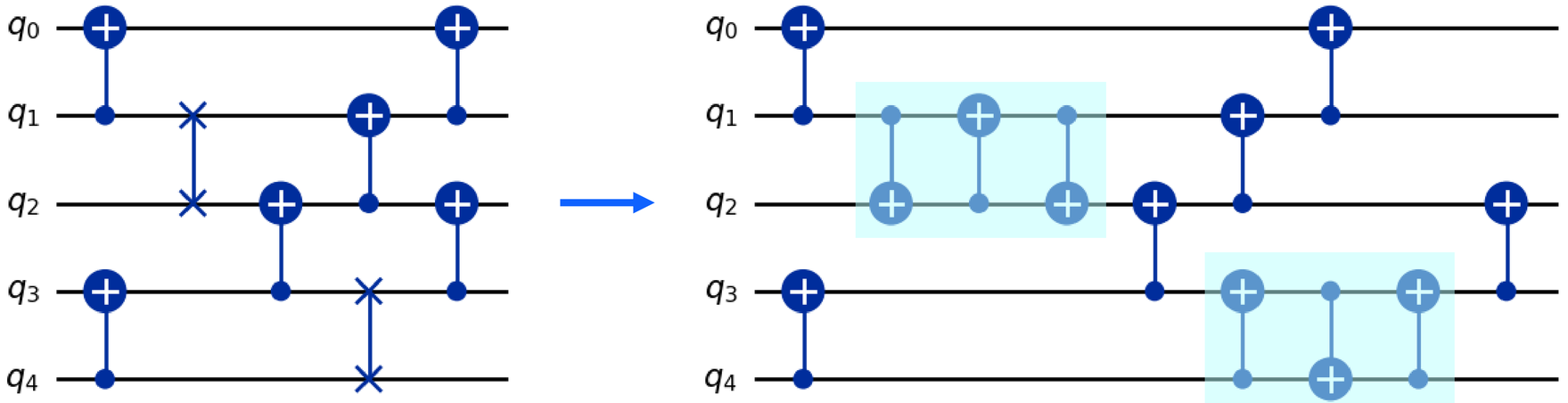
STAGED PASS MANAGER



Translation stage

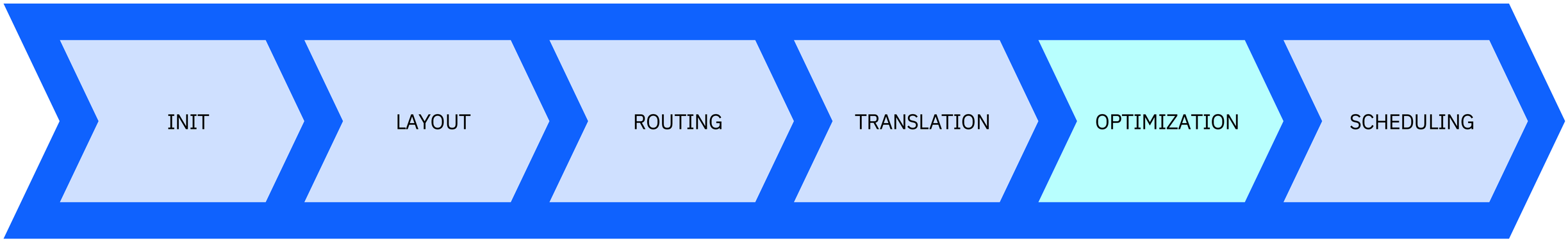
At this point we can translate all remaining operations into the hardware's ISA.





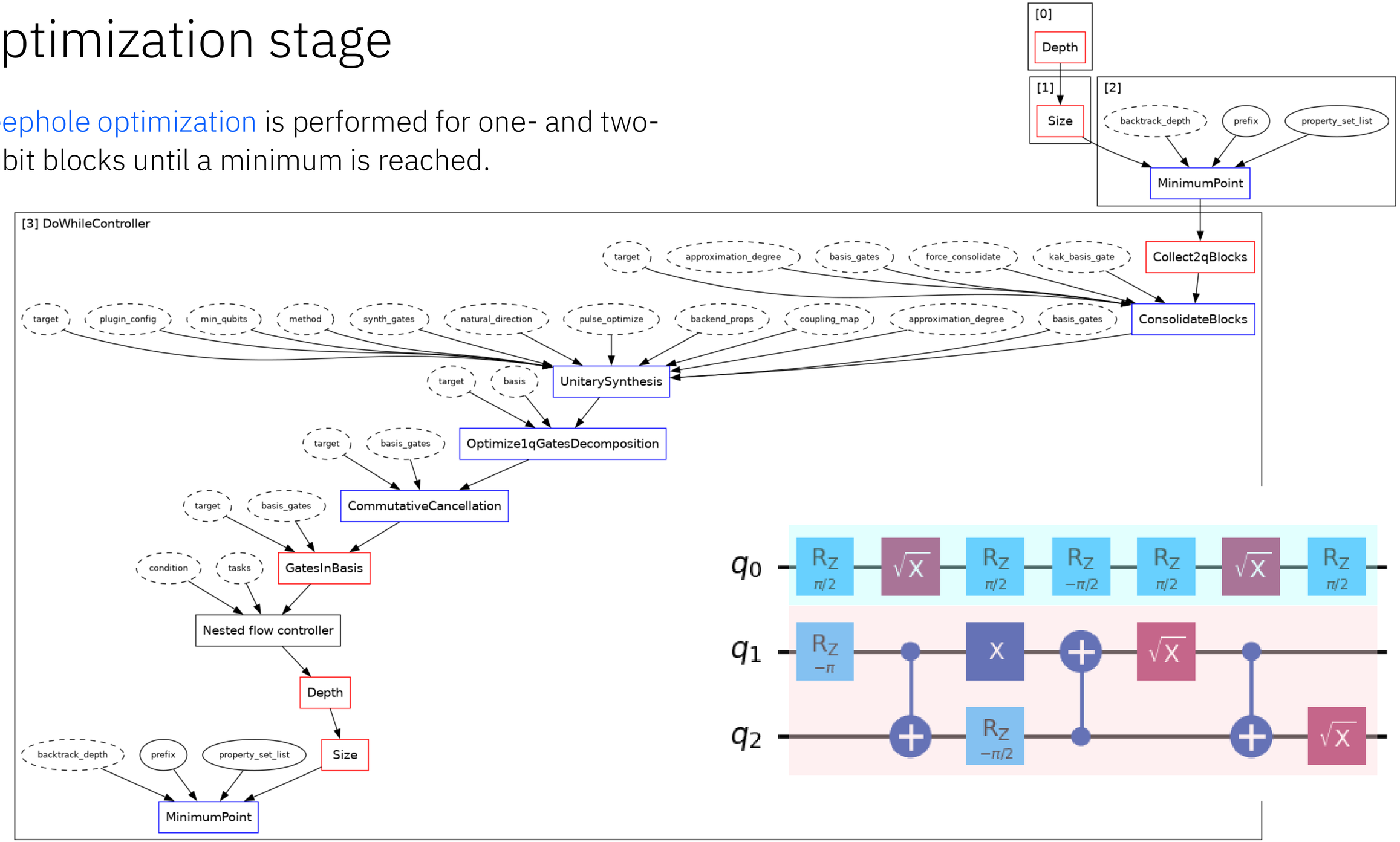
Transpiler stages

STAGED PASS MANAGER



Optimization stage

Peephole optimization is performed for one- and two-qubit blocks until a minimum is reached.



Optimization levels

Level 0: *[No Optimization]* - Basic translation with trivial qubit mapping and no optimizations, primarily for hardware characterization.

- Layout/Routing: TrivialLayout, where it selects the same physical qubit numbers as virtual and inserts SWAPs to make it work (using SabreSwap)

Level 1: *[Light Optimization]* - Reduces gate count and simplifies the circuit with minimal compile time, using basic layout and gate optimization.

- Layout/Routing: Layout is first attempted with TrivialLayout. If additional SWAPs are required, a layout with a minimum number of SWAPs is found by using SabreSwap, then it uses VF2LayoutPostLayout to try to select the best qubits in the graph.
- InverseCancellation
- 1Q gate optimization

Level 2: *[Medium Optimization]* - Applies more sophisticated heuristic-based optimizations to further reduce circuit complexity.

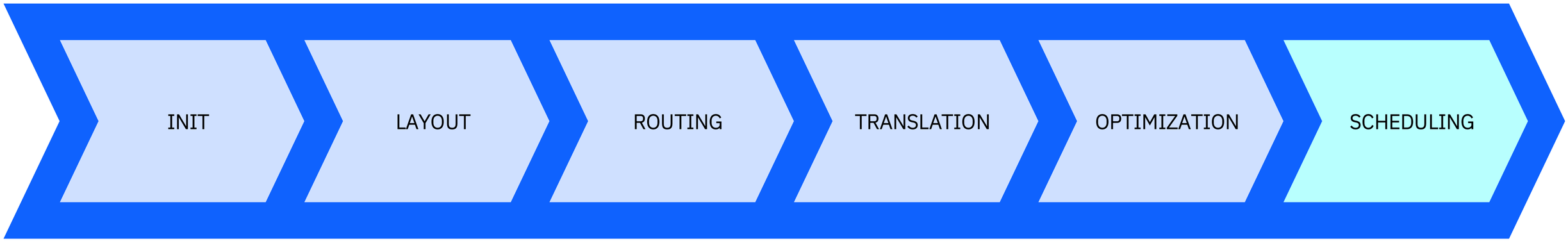
- Layout/Routing: Optimization level 1 (without trivial) + heuristic optimized with greater search depth and trials of optimization function. Because TrivialLayout is not used, there is no attempt to use the same physical and virtual qubit numbers.
- CommutativeCancellation

Level 3: *[High Optimization]* - Performs the most extensive optimizations, including advanced gate resynthesis and in-depth layout adjustments.

- Optimization level 2 + heuristic optimized on layout/routing further with greater effort/trials
- Resynthesis of two-qubit blocks using Cartan's KAK Decomposition.
- Unitarity-breaking passes:
- OptimizeSwapBeforeMeasure: Moves the measurements around to avoid SWAPs
- RemoveDiagonalGatesBeforeMeasure: Removes gates before measurements that would not effect the measurements

Transpiler stages

STAGED PASS MANAGER



(Optional)

Operator backpropagation (OBP) Addon (*hands-on*)

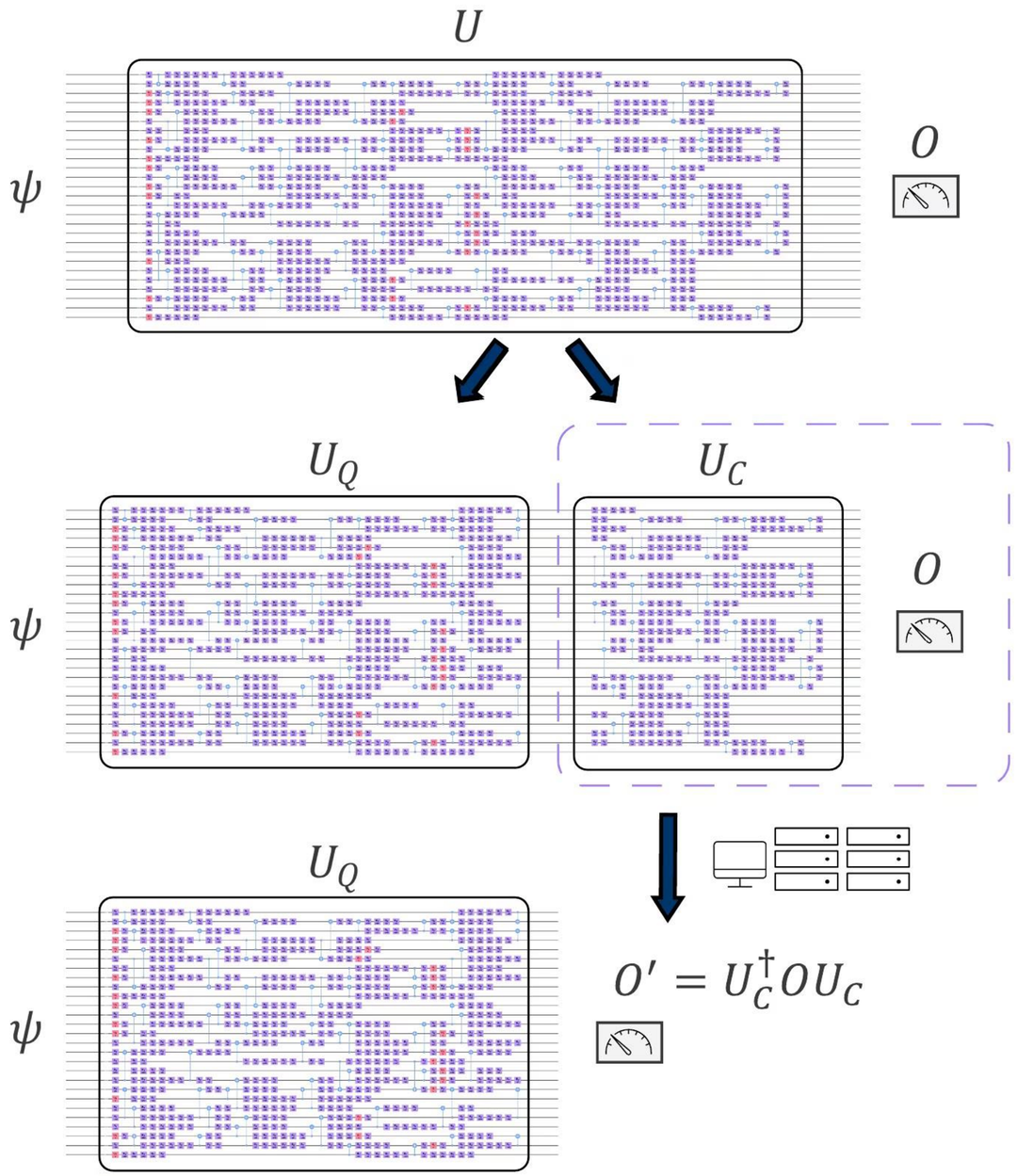
General Idea:

Trim circuit at the end to reduce circuit depth and simulate it classically, adapting the observables.

- Only works with Observables (not with Sampler)
- Needs a classically simulable subcircuit *Uc*
- Uses Clifford perturbation theory

Tradeoffs:

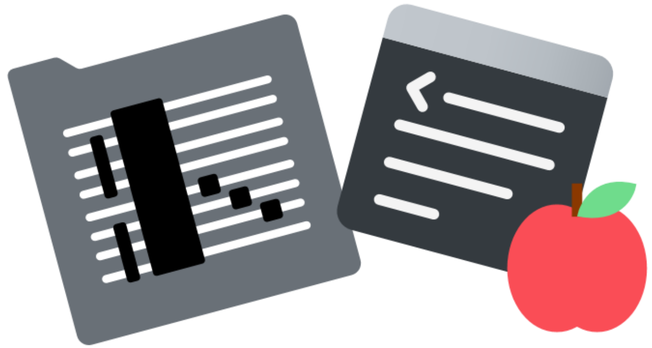
- Tradeoff between precision and efficiency
- Lower depth, but more operator measurements
- Size of the observable to measure grows exponentially with the amount of trimming.
- The *less* non-Clifford operations used, the more efficient is the simulation.
- We can increase efficiency by truncating observables with small coefficients.
- The error incurred can be controlled to find the best suited tradeoff



Transpilation best practices

- Don't be afraid of experimenting: use and build plugins, try the new AI transpilation service.
- Inspect circuits after transpilation to search for more ways of optimizing.
- Transpile the original circuit multiple times and save the best result to increase your chances of getting a better stochastic outcome.
- Select the extent of optimization based on your problem, keeping in mind the trade-off between computational cost and accuracy.
- Peephole optimization: if there are a lot of gates appearing on relatively few qubits numerical optimizations for the block could be beneficial.
- Specialized methods: large networks of Clifford gates can be further optimized.
- Operator Backpropagation (OBP): Remove big Clifford sections at the end of the circuit and permute the observable through. This could increase the weight of the observable but reduce the overall observed noise.
- Approximate quantum compiling (AQC): if gates are imperfect, approximating unitaries in such way that the total number of gates is reduced might be beneficial; effectively trading off unitary for execution accuracy. This could be done in a peephole fashion, but as gates become better the strategy becomes less effective.

Further reading



Courses:

Running quantum circuits:
https://quantum.cloud.ibm.com/learning/en/courses/quantum-computing-in-practice/running-quantum-circuits

Tutorials:

Operator Backpropagation Qiskit Addon:

https://quantum.cloud.ibm.com/docs/en/guides/qiskit-addons-obp

https://quantum.cloud.ibm.com/docs/en/tutorials/operator-back-propagation

Circuit Cutting Qiskit Addon:

https://quantum.cloud.ibm.com/docs/en/guides/qiskit-addons-cutting

https://quantum.cloud.ibm.com/docs/en/tutorials/depth-reduction-with-circuit-cutting